

04 — Persistence

OS, disk, and memory-only persistence — detection focus

Educational / Defensive DFIR Study Notes

LEARNING OBJECTIVES

- Enumerate major Windows persistence classes and their artifacts
- Map 4698, 7045, and Autoruns findings to proof workflows
- Distinguish reboot-surviving vs memory-only persistence
- Always record cmdline, path, time, and account for each finding

SCOPE REMINDER

- This material covers detection, artifacts, and incident response only.
- Attacker actions are phrased as 'adversaries may...' then pivot to evidence.
- Tool/command names appear as IOC strings / exam knowledge — not how-to guides.

Persistence classes

Class	Examples	Survives reboot?
OS / autostart	Service, scheduled task, Run/RunOnce, Startup folder, WMI subscription	Yes (if written correctly)
Disk implants	Service EXE, DLL sideload pair, dropped EXE in Startup, modified LNK	Yes
Memory-only	Reflective DLL / process injection without disk write	No — gone after reboot

Memory-only techniques are why IR acquires **memory before rebooting** when feasible. Disk persistence is why Autoruns + registry + task/service inventories matter after reboot.

Registry Run keys

- HKLM\Software\Microsoft\Windows\CurrentVersion\Run (and RunOnce) — machine-wide
- HKCU\...\Run (and RunOnce) — per user (triggers on that user's logon)
- Adversaries may add values pointing to dropped payloads or living-off-the-land binaries with odd arguments
- Prove: hive export / RECmd / Sysmon registry events; note value name, data, LastWrite time, user hive owner

Scheduled tasks

- Creation evidence: Security Event **4698** (Scheduled task created); also Task Scheduler Operational log
- Enumeration: schtasks, Task Scheduler GUI, Autoruns 'Scheduled Tasks'
- Read the task XML: action (exec/comhandler), arguments, run-as account, triggers, hidden flags
- Adversaries may name tasks to blend with software updaters — judge by path + hash + creation time vs installers

Services

- Installation evidence: System Event **7045** — 'A service was installed on the system'
- Also: 4688 for sc create / PowerShell service cmdlets; Services registry keys under CurrentControlSet
- Inspect ImagePath, ServiceDLL, Start type, account (LocalSystem is high impact)
- Prove malice via unexpected path (Temp, Users\Public), unsigned binary, rare hash

WMI persistence

Classic pattern: **Event Filter** + **Consumer** + **Filter-to-Consumer Binding**. Adversaries may trigger on frequent events (e.g., timer or process start) to launch a payload. Detect with WMI Explorer / Autoruns WMI view / Sysmon 19–21 (if configured) / wmicprvse.exe spawning unusual children.

How do I prove persistence?

Mechanism	Primary proof	Always record
Run key	Registry value + LastWrite; Sysmon 13 if available	cmdline/path, time, account hive
Scheduled task	4698 + task XML action	cmdline/path, time, run-as account
Service	7045 + ImagePath	cmdline/path, time, service account
WMI	Filter+Consumer+Binding triad	consumer command, filter query, time
Startup folder	File in Startup + MFT times + LNK targets	path, hash, creating process
Memory-only	malfind / EDR injection alerts pre-reboot	process, RWX region, network

Investigation tools

- **Autoruns / AutorunsSvc** — broad sweep; compare to known-good baseline
- Registry Explorer / RECmd, schtasks, services.msc / sc.exe query, WMI Explorer
- Timeline: combine 4698/7045/Sysmon with MFT for dropper write times

Common exam traps

- Assuming reboot clears the threat (disk persistence remains)
- Ignoring HKCU Run keys when the user still logs in
- Reading service Name but not ImagePath
- Missing WMI because it is not in 'Run' keys
- Forgetting the four-tuple: cmdline, path, creation time, account

Lab takeaways

FROM LAB 2-3 — PERSISTENCE

- Four mechanisms → one payload: WMI 5861/5859, task 4698/Operational 129, BITS re-fetch, Run key.
- Public staging path pattern (e.g. cdbupd.exe). BITS IP uncorroborated until second source.
- Registered vs fired: parent of 4688 (WmiPrvSE/taskhostw vs powershell).
- Absence of malicious service is a finding; remove all layers together.

QUICK REFERENCE

- 4698 = task created; 7045 = service installed
- Run/RunOnce: HKLM and HKCU
- WMI = Filter + Consumer + Binding
- Memory-only dies on reboot — image RAM first
- Autoruns + Event IDs + timeline = prove it

Mini Self-Check

Answer these without looking back. Answers are on the next page.

- Q1.** Which Event ID indicates a scheduled task was created?
- Q2.** Which Event ID indicates a service was installed?
- Q3.** Name the three WMI components used together for persistence.
- Q4.** Why might a host look 'clean' after reboot yet still be compromised?
- Q5.** List the four fields you should always record for a persistence finding.

Self-Check Answers

A1. 4698.

A2. 7045.

A3. *Event Filter, Consumer, and Filter-to-Consumer Binding.*

A4. *Disk-based persistence (service/task/Run key/WMI/startup file) survives reboot; only memory-only implants are cleared.*

A5. *Command line, path, creation time, and account context.*